Steve Freeman, Nat Pryce, and their co-authors refer to this as "Mock Roles, Not Objects" in their paper of the same name.[3] Steve and Nat explore related ideas in their book, *Growing Object-Oriented Software, Guided by Tests [FP09]*. For more on the importance on mocking roles rather than objects, see Gregory Moeck's RubyConf 2011 talk, "Why You Don't Get Mock Objects."[4]

Dependency Injection

To refactor the class to depend on abstract roles, we can use dependency injection. We first encountered this concept in *Connecting to Storage*, on page 65. The technique can take a few different forms, the simplest of which is *argument injection*:

15-using-test-doubles-effectively/15/api_request_tracker/lib/api_request_tracker.rb

```
class APIRequestTracker
  def process(request, reporter: MetricsReporter)
    endpoint_description = Endpoint.description_of(request)
    reporter.increment("api.requests.#{endpoint_description}")
  end
end
```

Now that the `process` method accepts an additional `reporter` argument, our test can easily inject a double:

15-using-test-doubles-effectively/15/api_request_tracker/spec/api_request_tracker_spec.rb

```
it 'increments the request counter' do
  reporter = class_double(MetricsReporter)
  expect(reporter).to receive(:increment).with('api.requests.get_users')

  APIRequestTracker.new.process(request, reporter: reporter)
end
```

This technique is simple and versatile. The main drawback is repetition. If you need to use the same collaborator from multiple methods, adding the same extra parameter to all of them can be cumbersome. Instead, you can use *constructor injection* to pass in your collaborator as part of the object's initial state:

15-using-test-doubles-effectively/16/api_request_tracker/lib/api_request_tracker.rb

```
class APIRequestTracker
  def initialize(reporter: MetricsReporter.new)
    @reporter = reporter
  end
```

3. http://www.jmock.org/oopsla2004.pdf
4. https://www.youtube.com/watch?v=R9FOchgTtLM